TEAM 10 · TSM_MACHLEDATA · ZHAW SPRING 2026

Is this image real?

An end-to-end MLOps system for AI-generated image detection — from Kaggle training to production-grade inference.

Team 10 / Marcos Costa, Artemii Ponomarenko, Afshin Khosroshahi

EfficientNet-B0 · ONNX · FastAPI · DVC · MLflow

github.com/495temych/ai-image-detector

The Problem

MOTIVATION & SCOPE

WHY IT MATTERS

- Generative AI has made photorealistic fake images trivially cheap to produce
- Misinformation, fraud, and synthetic media manipulation are accelerating
- Human visual inspection is no longer reliable — even experts fail at scale
- A deployable, explainable detector is needed — not just a research notebook

OUR APPROACH

```
INPUT → any image (URL or upload)
MODEL → EfficientNet-B0, fine-tuned
OUTPUT → Real / AI-generated + confidence
EXPLAIN→ GradCAM heatmap overlay
```

Full MLOps lifecycle: versioned data, tracked experiments, gated model registry, automated CI/CD, containerised serving.

03 Dataset & Data Versioning

SOURCE · DVC · DAGSHUB

KAGGLE CIFAKE DATASET

60k	20k	50 / 50
TRAINING IMAGES	TEST IMAGES	REAL / FAKE SPLIT

Real: CIFAR-10 photographs · Fake: Stable Diffusion v1.4
5 000-image subset used for fine-tuning in training runs

DVC DATA FLOW

01	*Kaggle API*	download raw dataset once → `data/raw/`
02	*dvc add*	hash & track `data/` + `models/` by MD5
03	*git commit*	`.dvc` pointer files committed to Git
04	*dvc push*	DagsHub remote · HTTPS · no S3
05	*dvc pull*	CI restores exact versioned artefacts

Remote: https://dagshub.com/marcosncosta1/ai-image-detector.dvc

Model

EFFICIENTNET-B0 · PYTORCH · GRADCAM

ARCHITECTURE

- EfficientNet-B0 pre-trained on ImageNet (transfer learning)
- Final classifier replaced: 1 280 → 512 → 1, sigmoid output
- Input: 224 × 224 RGB, standard ImageNet normalisation
- Trained on Kaggle GPU (T4) — 5 epochs, AdamW, OneCycleLR

EXPLAINABILITY — GRADCAM

- Gradient-weighted Class Activation Maps on last conv layer
- Heatmap overlaid on original image at inference
- PyTorch retained alongside ONNX specifically for backprop

INFERENCE PATHS

Fast Path — ONNX

```
# /predict endpoint
ONNX Runtime → ~0.1s
CPU-only · Docker-friendly
```

Explain Path — PyTorch

```
# /predict-explain endpoint
Full backprop → ~1s
GradCAM heatmap returned
```

05

Experiment Tracking

MLFLOW · DAGSHUB · MODEL REGISTRY

MLFLOW INTEGRATION

- Every training run logs: loss curves, accuracy, AUC-ROC, F1, confusion matrix
- Hyperparameters captured: LR, batch size, epochs, optimiser, scheduler
- Model artefacts registered to MLflow Model Registry on DagsHub
- Registry gate: accuracy ≥ 85% required for Staging → Production promotion

CHAMPION RUN

```
Run: efficientnet_v1
ID: 2561d86a…
Stage: Production
```

KEY RESULTS

89.6%	96.2%	0.90
ACCURACY	AUC-ROC	F1 SCORE

MLflow tracking URI:
https://dagshub.com/marcosncosta1/ai-image-detector.mlflow

06 # Pipeline / Flow Map

END-TO-END MLOPS

01

Kaggle

data source · CIFAKE dataset
GPU training notebook (T4)

02

DVC

data versioning · MD5 hashed
DagsHub remote (HTTPS)

03

PyTorch

model training · EfficientNet-B0
GradCAM backprop & explain

04

MLflow

metrics & params · artefacts
model registry (gated ≥ 85%)

05

GitHub Actions

CI on push(main) · 4 jobs
evaluate → register → export → docker

06

ONNX

portable export · CPU runtime
fast inference ~0.1 s

07

FastAPI + Docker

REST serving · /predict & /predict-explain
multi-platform push to GHCR

08

Node.js UI

Detector & Challenge modes
SQLite persistence · community stats

07

Components / Entered into Evidence

TOOLS & TECHNOLOGIES

DVC

DATA VERSIONING

MD5-hashed data & model snapshots. DagsHub HTTPS remote. dvc pull in CI.

Git

SOURCE CONTROL

Pointer files (.dvc) committed alongside code. GitHub Actions triggers.

PyTorch

TRAINING & GRADCAM

EfficientNet-B0 fine-tuning. Retained at inference for explainability backprop.

MLflow

EXPERIMENT TRACKING

Metrics, params, artefacts. Registry gate: accuracy ≥ 85% → Production.

GitHub Actions

CI / CD

4-job ML pipeline + lint/test. Triggers on push to main. Multi-platform build.

ONNX

MODEL EXPORT

Portable runtime-agnostic format. CPU inference ~0.1 s. No GPU needed in prod.

FastAPI

SERVING LAYER

/predict (ONNX) · /predict-explain (GradCAM). Auto-docs. Health check.

Docker

CONTAINERISATION

Multi-stage builds. linux/amd64 + linux/arm64. Images pushed to GHCR.

Node.js

WEB UI

Express + vanilla JS. Detector mode & Challenge mode. SQLite persistence.

GitHub

CODE HOSTING

Repository, PR workflow, Actions secrets, GHCR image registry.

Kaggle

DATA & TRAINING

CIFAKE dataset source. GPU notebook environment (T4) for training runs.

DagsHub

ML PLATFORM

DVC remote storage + MLflow tracking UI. Central collaboration hub.

08 CI / CD Pipeline

GITHUB ACTIONS · 4 ML JOBS + 2 QA JOBS

ML PIPELINE — ML-PIPELINE.YML

01	*Evaluate*	dvc pull → run evaluation → log metrics to MLflow
02	*Register*	accuracy ≥ 85% gate → promote to Production in registry
03	*Export*	PyTorch → ONNX export → push model artefact to DagsHub
04	*Docker*	build + push API & UI images to GHCR (amd64 + arm64)

QA PIPELINE — CI.YML

A	*Lint*	ruff check src/ api/ tests/ — every push & PR
B	*Test*	pytest tests/ — lightweight deps only, no GPU required

TRIGGER & GATES

```
on: push(main), pull_request(main)
concurrency: cancel-in-progress

# ML pipeline gate
if accuracy < 0.85: exit(1)

# Docker only on merge to main
if: github.ref == 'refs/heads/main'
```

Images tagged with both :latest and :sha
Multi-platform: linux/amd64 · linux/arm64
Registry: ghcr.io/marcosncosta1/

09

FastAPI Serving Layer

REST API · ONNX · GRADCAM

ENDPOINTS

GET	/health	model loaded, ONNX session, uptime
POST	/predict	image → label + confidence · ONNX path · ~0.1s
POST	/predict-explain	image → verdict + base64 GradCAM heatmap · ~1s

RESPONSE SCHEMA

```
{
  "label": "AI-generated",
  "confidence": 0.94,
  "heatmap_b64": "<base64 PNG>"
}
```

CONTAINER SETUP

```
# api/Dockerfile (multi-stage)
FROM python:3.11-slim
EXPOSE 8000
# model loaded at startup via
# lifespan() + DVC pull in CI
```

PyTorch retained alongside ONNX for GradCAM backprop — no GPU required in production container. Model path injected via env var.

Web UI — Detector & Challenge

NODE.JS · EXPRESS · SQLITE

DETECTOR MODE

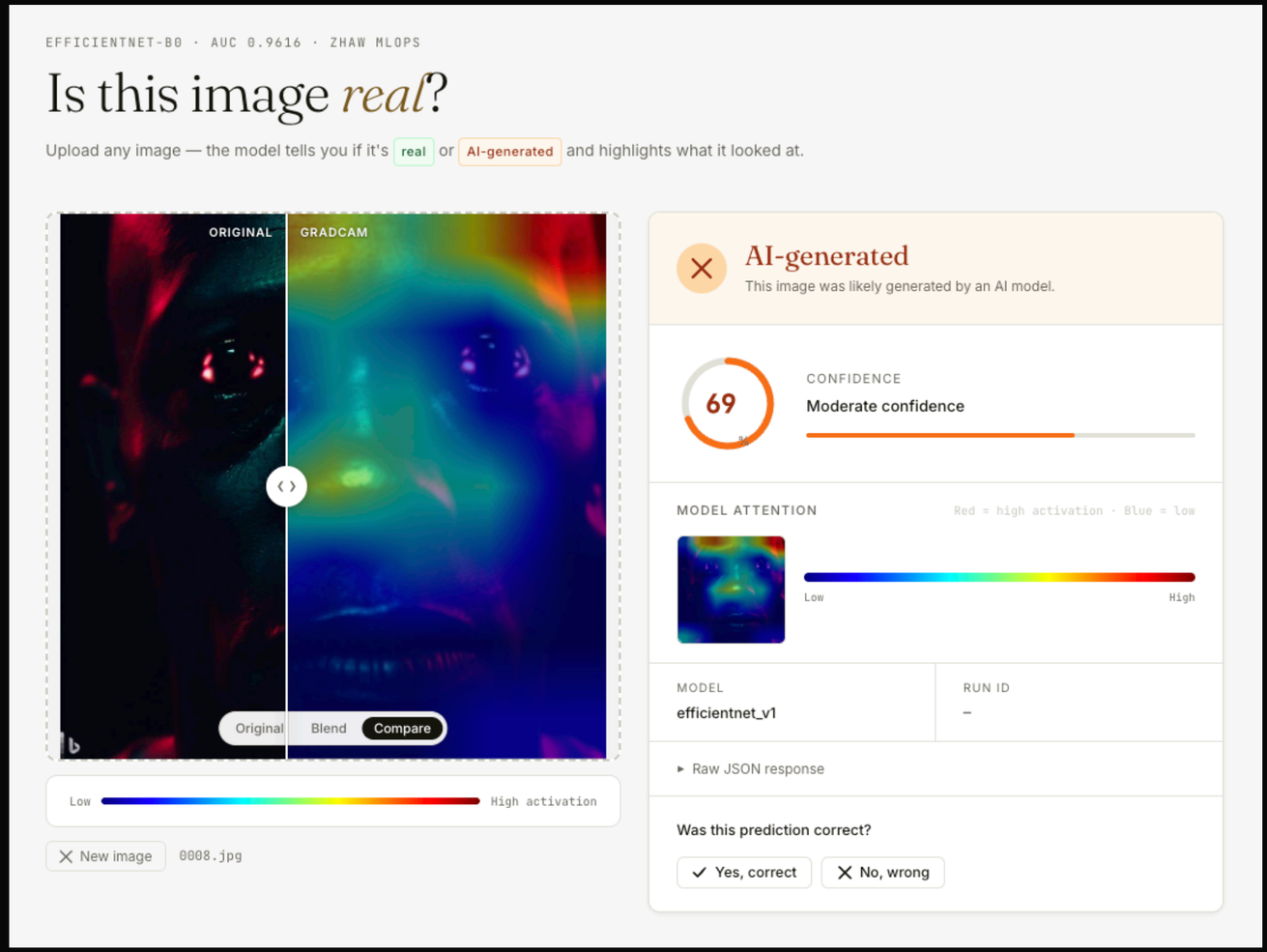
- Upload image or paste URL
- Calls `/predict-explain` → verdict + GradCAM overlay
- Heatmap shows which pixels triggered the decision

```
Verdict: AI-generated  ·  Confidence: 94.3%
# GradCAM heatmap rendered inline
```

CHALLENGE MODE

- 10-image quiz — guess Real or Fake
- Model score computed in parallel for same images
- Results saved to SQLite, shown in leaderboard

```
You: 7 / 10  ·  Model: 9 / 10
# stored in ui/data/scores.db
```



Community Analytics

LEADERBOARD · SQLITE · HUMAN VS MODEL

WHAT GETS TRACKED

- Every challenge session: timestamp, human score, model score, per-image results
- Aggregate win-rate: how often humans beat the model
- Most-confusing images flagged by repeated human errors
- Historical trend — is human accuracy improving over sessions?

STORAGE

```
SQLite · file: ui/data/scores.db
# Docker volume-mounted for persistence
volumes: ./ui/data:/app/data
```

SAMPLE STATS

68%	91%	12%
HUMAN AVG SCORE	MODEL AVG SCORE	HUMAN WIN-RATE

Humans consistently underperform the model on AI-generated nature scenes and portraits. Artefacts (blur, fingers) are the most reliable human tells.

Docker & Deployment

GHCR · DOCKER-COMPOSE · ONE-COMMAND DEPLOY

CONTAINER ARCHITECTURE

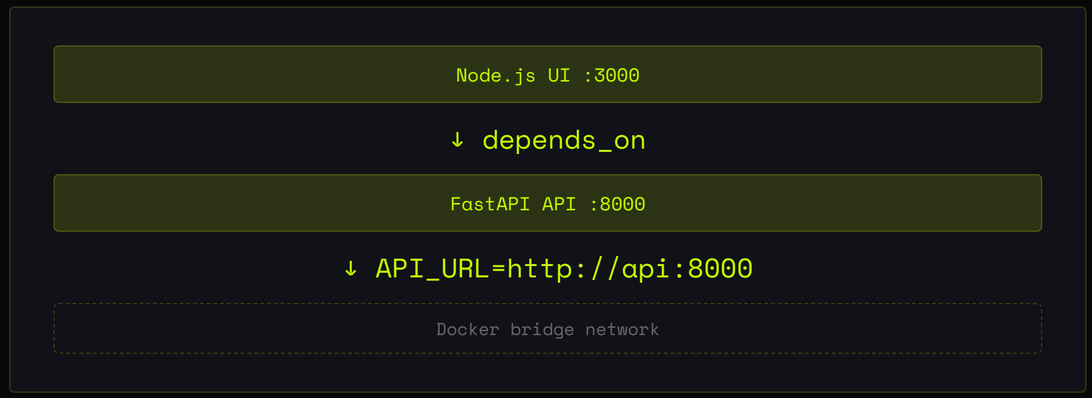
- Two services: `api` (FastAPI) and `ui` (Node.js)
- Both images built for linux/amd64 + linux/arm64 (Apple Silicon ready)
- Pushed to GHCR: `ghcr.io/marcosncosta1/`
- SQLite DB persisted via named volume across restarts

ONE-COMMAND DEPLOYMENT

```
# docker-compose.prod.yml
curl -O .../docker-compose.prod.yml
docker compose -f docker-compose.prod.yml up -d

# UI → :3000 · API → :8000
```

SERVICE GRAPH



No external database needed. SQLite lives on host via `./ui/data:/app/data` volume mount.

Results & Evaluation

MODEL PERFORMANCE · MLOPS OUTCOMES

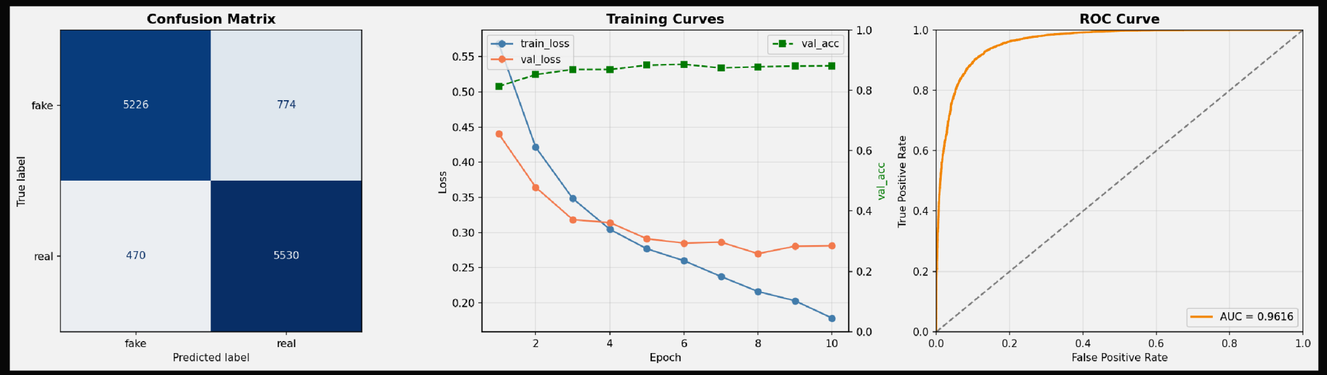
89.6%	96.2%	0.90	~0.1s
ACCURACY	AUC-ROC	F1 SCORE	INFERENCE (ONNX)

MLOPS OUTCOMES

- Fully reproducible: any commit restores exact data + model via DVC
- Zero-touch deploy: push to main → Docker images auto-published
- Model gating prevents regressions from reaching production

LIMITATIONS & FUTURE WORK

- CIFAKE is low-resolution (32×32 upscaled) — real-world images differ
- Next: retrain on DALL-E 3 & Midjourney for broader generator coverage



TEAM 10 · ZHAW SPRING 2026

Live Demo

Upload an image — real or AI-generated — and let the model decide. Then take the challenge: can you beat it?

REPOSITORY github.com/marcosncosta1/ai-image-detector

DEPLOY docker compose -f docker-compose.prod.yml up

EfficientNet-B0 · ONNX · FastAPI · DVC · MLflow
Node.js · Docker · GitHub Actions